

Authentication & Authorization

Lab Guide: securing the Personal Banking API with password hashing, JWT, roles and ownership checks

Item	Details
Builds on	Extension 5 reference solution (API design practices: versioning, ledger, pagination, errors, idempotency)
You receive	<code>banking_api_ext6_starter.zip</code> (code with TODOs + 108 acceptance tests)
Stack	Python 3.10+, FastAPI, SQLAlchemy, PyJWT, pwdlib (Argon2id), pytest
Time	About 2.5 hours for Tasks 1–6, plus optional stretch goals
Definition of done	<code>pytest -v</code> shows 108 passed (51 existing + 57 new)
If you get stuck	<code>Ext6_Unstuck_Hints.pptx</code> : three hint levels per task, then the code

Contents

- 1** The story
- 2** Setup (15 minutes)
- 3** The security contract
- T1** Users and safe password storage
- T2** Login issues a JWT
- T3** Who is calling? `get_current_user`
- T4** Lock the doors
- T5** Is it yours? Ownership checks (BOLA)
- T6** Roles (RBAC)
- S** Stretch goals
- ✓ Submission checklist and reflection
- A–C** Appendices: manual walkthrough, expected results, troubleshooting

1. The story

Our Personal Banking API is now well designed: versioned under `/api/v1`, every money movement lands in a ledger, transaction history can be filtered and paginated, errors share one envelope, and transfers are safe to retry. There is one big gap. **Every caller is anonymous.** Anyone who can reach the server can read any balance, withdraw from any account, and mark their own account as KYC compliant.

In this lab you close that gap the way real banking backends do. By the end, the API will answer three questions on every request:

Question	Mechanism you build	Failure	Task
Who are you?	Login issues a signed JWT; <code>get_current_user</code> verifies it	401 Unauthorized	1, 2, 3, 4
Is it yours?	<code>accounts.owner_id + AccountAccess.view() / operate()</code>	404 Not Found	5
Is your role allowed?	<code>require_role("ADMIN")</code> dependency	403 Forbidden	6

Learning objectives

- Store passwords with a salted, slow hash and apply NIST SP 800-63B-4 length rules.
- Issue short-lived JWTs with minimal claims and verify them with a pinned algorithm.
- Write FastAPI dependencies for authentication and apply them at router and route level.
- Enforce object-level authorization (OWASP API1:2023 BOLA) and role-based access control.
- Choose deliberately between 401, 403 and 404, and keep the Extension 5 error envelope.
- Keep existing behaviour working: all 51 Extension 5 tests must still pass.

2. Setup (15 minutes)

1. Unzip `banking_api_ext6_starter.zip` and open the `banking_api` folder in your IDE.
2. Create and activate a virtual environment, then install the dependencies: `pip install -r requirements.txt`. Three packages are new: `PyJWT`, `pwlib[argon2]` and `python-multipart`.
3. Delete any old `bank.db` from Extension 5. The `accounts` table gets a new column in Task 5, and `create_all()` never alters an existing table. (In production you would write an Alembic migration instead.)
4. Open `.env` and read the new settings. Generate your own secret and paste it into `JWT_SECRET_KEY: python -c "import secrets; print(secrets.token_urlsafe(64))"`
5. Start the server: `uvicorn app.main:app --reload`, then open `http://127.0.0.1:8000/docs`. You will see new **Auth** and **Admin** sections and an **Authorize** button.
6. Run the test suite once: `pytest -q`. Expect many failures and errors. That is your to-do list.
7. List every place you must edit: `grep -rn "TODO" app/` (Windows PowerShell: `Select-String -Path app\*\*.py -Pattern TODO`).

Why do the Extension 5 tests fail too?

The shared `client` fixture in `tests/conftest.py` now logs in as a default customer before each test. Until registration and login work (Tasks 1 and 2), that fixture cannot get a token. Work task by task with `pytest tests/test_auth1_users.py -v` and the old tests turn green again as you go.

What is provided and what you build

File	Status	Notes
<code>app/config.py</code> , <code>.env</code>	Provided	<code>JWT_SECRET_KEY</code> (required, ≥ 32 chars), algorithm, issuer, 15-minute expiry, bootstrap admin
<code>app/models/user.py</code>	Provided	id, username (unique), hashed_password, role, is_active, created_at
<code>app/repositories/user_repository.py</code>	Provided	get_by_id, get_by_username (lower-case), list_all, create, save
<code>app/core/errors.py</code>	Provided	<code>NotAuthenticatedError</code> (401 + WWW-Authenticate), <code>ForbiddenError</code> (403), <code>UsernameTakenError</code> (409)
<code>app/main.py</code> , <code>app/api/v1/__init__.py</code>	Provided	Auth + admin routers wired under <code>/api/v1</code> ; admin seeded at startup
<code>app/core/security.py</code>	You build	Tasks 1–3: hashing, token creation, token decoding
<code>app/schemas/auth_schema.py</code>	You build	Task 1: validation rules, safe output schema
<code>app/services/auth_service.py</code>	You build	Tasks 1–2: register, login
<code>app/routers/auth_router.py</code>	You build	Tasks 2–3: <code>/auth/token</code> , <code>/auth/me</code>

File	Status	Notes
<code>app/core/auth.py</code>	You build	Tasks 3, 5, 6: <code>get_current_user</code> , <code>AccountAccess</code> , <code>require_role</code>
account / transaction / scheduled-payment routers	You modify	Tasks 4–6: protect routes, apply ownership and roles
<code>app/models/account.py</code> , account service + repository	You modify	Task 5: <code>owner_id</code> and "my accounts only"
<code>app/routers/admin_router.py</code>	You modify	Task 6: make the whole router ADMIN-only

3. The security contract

This section is the specification. The acceptance tests check exactly these rules, so keep it open while you work.

3.1 New endpoints

Method & path	Auth	Success	Notes
POST <code>/api/v1/auth/register</code>	Public	201 + UserOut	JSON <code>{username, password}</code> ; role is always CUSTOMER
POST <code>/api/v1/auth/token</code>	Public	200 + Token	FORM data <code>username=..&password=..</code> (OAuth2 password flow)
GET <code>/api/v1/auth/me</code>	Any user	200 + UserOut	Who does the server think I am?
GET <code>/api/v1/admin/users</code>	ADMIN	200 list	Never includes password hashes
PATCH <code>/api/v1/admin/users/{id}</code>	ADMIN	200 + UserOut	<code>{"is_active": false}</code> locks a user out immediately

3.2 Token rules

Claim	Value	Why
<code>sub</code>	user id as a string , e.g. <code>"7"</code>	Standard subject claim; PyJWT ≥ 2.10 rejects non-strings
<code>role</code>	CUSTOMER or ADMIN	Hint for clients; the server re-reads the role from the database
<code>iss</code>	<code>bank_api</code>	Rejects tokens minted by other systems
<code>iat</code> / <code>exp</code>	now / now + 15 minutes	Short-lived tokens limit damage when stolen
Anything else	Not allowed	The payload is readable by anyone: no balance, account number, name or email

Algorithm: HS256, pinned on decode with `algorithms=["HS256"]`. Response shape: `{"access_token": "...", "token_type": "bearer", "expires_in": 900}`.

3.3 Authorization policy

Action	Owner	Other customer	Admin
View account, KYC status, transactions, scheduled payments	Yes	404	Yes
Deposit, withdraw, transfer FROM, create/patch/delete scheduled payment	Yes	404	403
Transfer TO an account	Anyone logged in	Anyone logged in	–
List accounts (GET <code>/accounts/</code>)	Own accounts only	Own accounts only	All accounts

Action	Owner	Other customer	Admin
Set KYC status (PUT .../kyc-status)	403	403	Yes
Admin endpoints	403	403	Yes

3.4 Status code rules

- **401** + header **WWW-Authenticate: Bearer**: no token, invalid or expired token, unknown or disabled user, wrong username or password. Always the same generic message.
- **403 forbidden**: the caller is known, but their role does not allow the action.
- **404 account_not_found**: the account does not exist OR belongs to someone else (for customers). Both look identical, so nobody can probe which ids exist.
- **409 username_taken**, **422** for validation errors (including an unexpected **role** field on register).
- All **/api/v1** errors keep the Extension 5 envelope: `{"error": {"code", "message", "details", "request_id"}}`.

Task 1: Users and safe password storage

Suggested time: 25 min · Tests: tests/test_auth1_*.py

Why this matters in a bank

If the users table ever leaks (backup left in a bucket, SQL injection, insider), plain or fast-hashed passwords turn one incident into thousands of compromised customers, many of whom reuse passwords elsewhere. A slow salted hash makes each guess expensive and each hash unique.

What you must do

1. In `app/core/security.py` implement `hash_password()` and `verify_password()` using the provided `_password_hash = PasswordHash.recommended()` (Argon2id).
2. In `app/schemas/auth_schema.py`, harden `UserCreate`: add `model_config = ConfigDict(extra="forbid")`; username 3–32 characters matching `^[A-Za-z0-9_.-]+`; password 15–128 characters.
3. Complete `UserOut` so it exposes `id`, `username`, `role`, `is_active`, `created_at`, and nothing else.
4. In `AuthService.register()`: lower-case the username, raise `UsernameTakenError` if it exists, otherwise save a `User` with `hashed_password=hash_password(...)` and `role=Role.CUSTOMER`.
5. Restart the server. The bootstrap admin from `.env` is now created at startup (it needed `hash_password`).

Acceptance criteria

- ☐ `POST /api/v1/auth/register` returns 201 with role `CUSTOMER`.
- ☐ The response contains neither the password nor the hash.
- ☐ The stored value starts with `$argon2` and is never the plain password.
- ☐ Hashing the same password twice gives two different hashes.
- ☐ Duplicate usernames (any letter case) return 409 `username_taken`.
- ☐ Passwords shorter than 15 characters return 422.
- ☐ Sending `"role": "ADMIN"` returns 422.

Verify

```
pytest tests/test_auth1_users.py -v          # expect 7 passed

curl -s -X POST localhost:8000/api/v1/auth/register \
  -H "Content-Type: application/json" \
  -d '{"username": "asha", "password": "Correct-Horse-Battery-9"}'
```

Check your understanding

- Why does the same password produce a different hash every time, and how can `verify_password` still work?

- What would happen if `UserCreate` silently ignored unknown fields and the service did `User(**payload.model_dump())`?
- Why is SHA-256 a bad choice even with a salt?

Task 2: Login issues a JWT

Suggested time: 25 min · Tests: tests/test_auth2_*.py

Why this matters in a bank

Login is where attackers aim credential-stuffing and brute-force attacks. The endpoint must not reveal which usernames exist (through messages or response time), and the token it returns must not leak customer data, because anyone who sees the token can read its payload.

What you must do

1. In `security.py` set `DUMMY_HASH = hash_password("any-long-dummy-value")`.
2. Implement `create_access_token(user_id, role)` with claims `sub` (string!), `role`, `iss`, `iat`, `exp` (`now + ACCESS_TOKEN_EXPIRE_MINUTES`), signed with `settings.JWT_SECRET_KEY` and `settings.JWT_ALGORITHM`.
3. Implement `AuthService.login()`: look the user up, ALWAYS verify exactly one hash (the user's or `DUMMY_HASH`), and raise `NotAuthenticatedError("Incorrect username or password")` for unknown user, wrong password or inactive user. Return `access_token`, `token_type="bearer"`, `expires_in` in seconds.
4. In `auth_router.py` add `POST /token` with form: `OAuth2PasswordRequestForm = Depends()` and `response_model=Token`.
5. Try it in Swagger: click **Authorize**, enter the admin username and password from `.env`. Swagger calls your `/token` endpoint.

Acceptance criteria

- ☐ Valid credentials return 200 with `token_type: bearer` and `expires_in > 0`.
- ☐ The token has three dot-separated parts and header `alg: HS256`.
- ☐ Claims contain `sub` (string), `role`, `iss = bank_api`, `iat`, `exp`, with a lifetime of 1 hour or less.
- ☐ No `password`, `balance`, `account_number` or `owner_name` in the claims.
- ☐ Wrong password and unknown user return the same 401 message and a `WWW-Authenticate: Bearer` header.
- ☐ JSON body on `/token` returns 422 (it expects form data).

Verify

```
pytest tests/test_auth2_login.py -v          # expect 4 passed

curl -s -X POST localhost:8000/api/v1/auth/token \
  -d "username=asha&password=Correct-Horse-Battery-9"
# copy access_token into a variable:
TOKEN=<paste token here>
# the payload is readable by anyone:
echo $TOKEN | cut -d. -f2 | base64 -d 2>/dev/null; echo
```

Check your understanding

- Why must login run a hash check even when the username does not exist?
- Why does the OAuth2 password flow use form data instead of JSON?
- If `role` is in the token, why does the server still read the role from the database?

Task 3: Who is calling? `get_current_user`

Suggested time: 30 min · Tests: `tests/test_auth3_*.py`

Why this matters in a bank

Every protected request trusts whatever `get_current_user` returns. One missing check (expiry, pinned algorithm, issuer) and forged or stolen tokens get in. Re-loading the user from the database means a disabled customer or a compromised account can be locked out immediately, without waiting for the token to expire.

What you must do

1. Implement `decode_access_token(token)` in `security.py` using `jwt.decode` with `algorithms=[settings.JWT_ALGORITHM]`, `issuer=settings.JWT_ISSUER` and `options={"require": ["exp", "iat", "sub", "iss"]}`. Let exceptions propagate.
2. Implement `get_current_user` in `app/core/auth.py`: no token raises `NotAuthenticatedError("Not authenticated")`; any `jwt.PyJWTError`, `ValueError` or `KeyError` while decoding raises `NotAuthenticatedError("Invalid or expired token")`; load the user with `int(claims["sub"])`; a missing or inactive user raises the same 401.
3. Add `GET /me` to `auth_router.py` returning `Depends(get_current_user)` with `response_model=UserOut`.
4. Notice that `oauth2_scheme` uses `auto_error=False`. That is why you raise your own 401, which goes through the Extension 5 handler and gets the standard envelope. The handler now also forwards `exc.headers`.

Acceptance criteria

- ☐ `GET /api/v1/auth/me` with a valid token returns the user (no hash).
- ☐ No token: 401, `WWW-Authenticate: Bearer`, error code `not_authenticated`, `request_id` present.
- ☐ Garbage token, tampered payload (role changed to ADMIN), wrong signing key: 401.
- ☐ Expired token, `alg: none` token, wrong issuer: 401.
- ☐ Valid signature for a user id that does not exist: 401.

Verify

```
pytest tests/test_auth3_current_user.py -v # expect 9 passed

curl -s localhost:8000/api/v1/auth/me -H "Authorization: Bearer $TOKEN"
curl -si localhost:8000/api/v1/auth/me | head -5 # 401
```

Check your understanding

- Which single line stops the `alg: none` attack? What would the token header say?
- Why is it safe to return the same message for "expired" and "forged"? Who benefits from a precise message?

- Stateless tokens were sold as "no database lookup". What do we gain by doing one lookup anyway?

Task 4: Lock the doors

Suggested time: 20 min · Tests: tests/test_auth4_*.py

Why this matters in a bank

Security that relies on remembering to add a check to every new endpoint fails the first time someone forgets. Protecting the whole router makes the safe behaviour the default, so a forgotten check fails closed (401), not open.

What you must do

1. In `account_router.py` add `dependencies=[Depends(get_current_user)]` to the `APIRouter(...)`. This one router serves both `/accounts` (legacy) and `/api/v1/accounts`, so both are locked.
2. Do the same for `scheduled_payment_router.py`.
3. In `transaction_router.py` the single route will get an ownership dependency in Task 5, and that dependency already requires a user. For now add `dependencies=[Depends(get_current_user)]` to the route or the router.
4. Keep `/`, `/docs` and `/openapi.json` public. Keep `/auth/register` and `/auth/token` public (they are on a different router).
5. Run the full suite. The Extension 5 tests pass again (the `client` fixture sends the default customer's token), except `test_not_found_v1_and_legacy`, which expects the `account_not_found` code from the Task 5 ownership check.
6. One case in the Task 4 file, `GET /api/v1/admin/users`, only turns green in Task 6 when the admin router gets its role dependency.

Acceptance criteria

- ☐ All 17 protected routes in `test_auth4_protected_routes.py` return 401 with `WWW-Authenticate: Bearer` when no token is sent, including legacy `/accounts/`.
- ☐ `/`, `/docs`, `/openapi.json` return 200 without a token.
- ☐ The OpenAPI document declares an `oauth2` security scheme (the Authorize button).

Verify

```
pytest tests/test_auth4_protected_routes.py -v
# -> 20 passed now, 21 after Task 6
pytest tests/test_task*.py -q
# -> Extension 5: 50 passed now, 51 after Task 5
```

Check your understanding

- Locking the legacy `/accounts` paths breaks unauthenticated legacy clients. Why is that the right call here, and how do the Extension 5 `Deprecation/Sunset` headers help?
- What is the difference between `dependencies=[Depends(x)]` and a parameter `user = Depends(x)`?

Task 5: Is it yours? Ownership checks (BOLA)

Suggested time: 35 min · Tests: tests/test_auth5_*.py

Why this matters in a bank

Broken Object Level Authorization is the number one risk in the OWASP API Security Top 10 (2023). After Task 4, any logged-in customer can still read or drain any account just by changing the id in the URL. Most real API data breaches look exactly like this.

What you must do

1. Add `owner_id = Column(Integer, ForeignKey("users.id"), index=True, nullable=True)` to `Account`. Delete `bank.db` and restart.
2. `AccountService.create_account(owner_name, owner_id)` stores the owner. In the router, take `user: User = Depends(get_current_user)` and pass `owner_id=user.id`.
3. `AccountRepository.get_all(owner_id=None)` filters in SQL when `owner_id` is given. `list_accounts` passes the caller's id (admins are handled in Task 6).
4. Implement `AccountAccess.view(account_id)` and `.operate(account_id)` in `app/core/auth.py` following policy 3.3. Missing account or not yours: raise `AccountNotFoundError`. `operate()` by an ADMIN on someone else's account: raise `ForbiddenError`.
5. Apply the provided wrappers with `dependencies=[Depends(can_view_account)]` or `can_operate_account` on every route that has `{account_id}` in its path: get account, deposit, withdraw, GET kyc-status, transactions, create and list scheduled payments.
6. Transfer has no `account_id` in the path: inject `access: AccountAccess = Depends()` and call `access.operate(request.from_account_id)` first. The destination can belong to anyone.
7. Scheduled payments by `payment_id`: load the payment with the service, then call `access.view(payment.account_id)` (GET) or `access.operate(...)` (PATCH, DELETE).
8. Scope idempotency keys per user: store `f"user:{access.user.id}:{idempotency_key}"` so two users choosing the same key never see each other's cached response.

Acceptance criteria

- ☐ A new account belongs to its creator; `GET /accounts/` lists only the caller's accounts.
- ☐ Reading, depositing to or withdrawing from another customer's account returns 404 and changes nothing.
- ☐ Transferring FROM someone else's account returns 404; transferring TO it works.
- ☐ Another customer's transactions and scheduled payments (GET, PATCH, DELETE) return 404.
- ☐ The same Idempotency-Key used by two different users produces two independent transfers.

Verify

```
pytest tests/test_auth5_ownership.py -v      # expect 9 passed
```

```
# Manual BOLA test: log in as two users, create an account as user A,
```

```
# then GET /api/v1/accounts/{A's id} with user B's token -> 404
```

Check your understanding

- Why return 404 instead of 403 when a customer asks for someone else's account?
- Why does **AccountAccess** have two methods instead of one?
- Where else in a real bank API would an object id need the same check (cards, beneficiaries, statements...)?

Task 6: Roles (RBAC)

Suggested time: 25 min · Tests: tests/test_auth6_*.py

Why this matters in a bank

A customer who can certify their own KYC defeats the whole anti-money-laundering control. Some actions belong only to staff. At the same time, staff should not be able to move customer money: least privilege limits the damage a compromised admin account can do.

What you must do

1. Implement the `require_role(*allowed_roles)` factory in `app/core/auth.py`: the inner `checker` depends on `get_current_user` and raises `ForbiddenError` (with `required_roles` and `your_role` in details) when the role is not allowed.
2. Make `PUT /accounts/{account_id}/kyc-status` ADMIN-only with `dependencies=[Depends(require_role(Role.ADMIN))]`. Note that `tests/conftest.py` already certifies KYC with admin headers.
3. In `list_accounts`, pass `owner_id=None` when `user.role == Role.ADMIN`, so admins see every account.
4. Add `dependencies=[Depends(require_role(Role.ADMIN))]` to the `APIRouter` in `admin_router.py`.
5. Check that `AccountAccess.operate()` already returns 403 for admins (Task 5) and `view()` lets them read.

Finish line

Run `pytest -v`. You should see **108 passed**. Then walk through the manual scenario in Appendix A once, with Swagger or curl.

Acceptance criteria

- ☐ A customer setting KYC returns 403 `forbidden` and the flag stays false.
- ☐ An admin can set KYC (200).
- ☐ An admin sees accounts owned by other customers in `GET /accounts/`.
- ☐ An admin can view a customer account but gets 403 on withdraw.
- ☐ Customers get 403 on `/api/v1/admin/users`; admins get the list without hashes.
- ☐ After an admin sets `is_active: false`, the user's existing (unexpired) token returns 401.

Verify

```
pytest tests/test_auth6_roles.py -v      # expect 7 passed
pytest -v                                # FINISH LINE: 108 passed
```

Check your understanding

- Why is `require_role` a function that returns a function?
- Which test proves that re-loading the user in `get_current_user` was worth it?
- Should an admin be able to promote another user to ADMIN through the API? What controls would you add?

Stretch goals (optional)

Each stretch goal is independent. Write your own tests for them in a new file, `tests/test_auth_stretch.py`.

S1. Refresh tokens with rotation

- Add a `refresh_tokens` table (id, user_id, token hash, expires_at, revoked). Store only a HASH of the refresh token.
- `POST /auth/token` also returns a long-lived random `refresh_token` (7 days).
- `POST /auth/refresh` accepts it, revokes it, and issues a NEW access + refresh pair (rotation). Reusing an old refresh token revokes the whole family (theft detection).
- `POST /auth/logout` revokes the refresh token.

S2. Brute-force protection

- Count failed logins per username and per IP in a table or in memory.
- After 5 failures in 15 minutes, return 429 `too_many_requests` with a `Retry-After` header. The Extension 5 error handler already maps 429.
- Discuss: why is a hard account lockout risky (attackers can lock real customers out)?

S3. Audit trail

- Add `performed_by` (user id) to the `transactions` ledger and fill it in deposit, withdraw and transfer.
- Add an `audit_events` table recording logins, failed logins, KYC changes and user deactivation, with the `X-Request-ID` from Extension 5.

S4. Token revocation by jti

- Add a random `jti` claim. Keep a denylist table of revoked jti values until their `exp`.
- Check the denylist in `get_current_user`. Measure: how much latency does one extra lookup add?

Submission checklist

- ☐ `pytest -v` shows 108 passed (plus your own stretch tests, if any).
- ☐ No secret is hard-coded in Python files; `.env` is listed in `.gitignore` for your own repository.
- ☐ No endpoint response or log line contains a password or password hash.
- ☐ JWT payload holds only `sub`, `role`, `iss`, `iat`, `exp`.
- ☐ Every route with an account id or payment id in the path has an ownership check.
- ☐ You can explain, for any failing request, why it is 401, 403 or 404.
- ☐ Answers to the "Check your understanding" questions written in `ANSWERS.md`.

Reflection (discuss with your pair)

- Our API issues its own tokens. What changes if the bank moves login to an Identity Provider such as Keycloak or Entra ID? What stays the same?
- Which of today's checks would you move into an API gateway, and which must stay in the service? Hint: can a gateway know who owns account 42?
- A mobile app stores the access token. Where exactly should it be stored on iOS and Android, and why not in plain app storage?

Appendix A: end-to-end manual walkthrough

Run these commands in order with the server running (Git Bash, macOS or Linux shell). In Swagger, use the Authorize button instead of the **-H** header.

```
API=localhost:8000/api/v1
JSON="Content-Type: application/json"

# 1. Two customers register
curl -s -X POST $API/auth/register -H "$JSON" \
  -d '{"username":"asha","password":"Correct-Horse-Battery-9"}'
curl -s -X POST $API/auth/register -H "$JSON" \
  -d '{"username":"ravi","password":"Another-Long-Passphrase-1"}'

# 2. Log in (form data!) and keep the tokens
login() {
  curl -s -X POST $API/auth/token -d "username=$1&password=$2" |
  python -c "import sys,json;print(json.load(sys.stdin)['access_token'])"
}
ASHA=$(login asha Correct-Horse-Battery-9)
RAVI=$(login ravi Another-Long-Passphrase-1)
ADMIN=$(login bank.admin 'Admin-Passphrase-2026!')

# 3. Asha opens an account (note its id) and deposits
curl -s -X POST $API/accounts/ -H "$JSON" \
  -H "Authorization: Bearer $ASHA" -d '{"owner_name":"Asha"}'
ID=1      # <- use the id returned above
curl -s -X POST $API/accounts/$ID/deposit -H "$JSON" \
  -H "Authorization: Bearer $ASHA" -d '{"amount":500}'

# 4. Ravi tries to read and drain it -> 404, 404
curl -s $API/accounts/$ID -H "Authorization: Bearer $RAVI"
curl -s -X POST $API/accounts/$ID/withdraw -H "$JSON" \
  -H "Authorization: Bearer $RAVI" -d '{"amount":500}'

# 5. Asha tries to self-certify KYC -> 403; the admin can -> 200
curl -s -X PUT $API/accounts/$ID/kyc-status -H "$JSON" \
  -H "Authorization: Bearer $ASHA" -d '{"kyc_compliant":true}'
curl -s -X PUT $API/accounts/$ID/kyc-status -H "$JSON" \
  -H "Authorization: Bearer $ADMIN" -d '{"kyc_compliant":true}'

# 6. No token at all -> 401 with WWW-Authenticate: Bearer
curl -si $API/accounts/ | head -6
```

Appendix B: expected results cheat-sheet

Scenario	Expected
Any money route without a token	401 not_authenticated + WWW-Authenticate: Bearer
Expired / tampered / alg: none / wrong issuer token	401 "Invalid or expired token"

Scenario	Expected
Wrong password or unknown user on <code>/auth/token</code>	401 "Incorrect username or password" (identical)
Register with an existing username	409 <code>username_taken</code>
Register with <code>role</code> field or 10-char password	422 <code>validation_error</code>
Customer reads another customer's account	404 <code>account_not_found</code>
Customer sets KYC	403 <code>forbidden</code>
Admin withdraws from a customer account	403 <code>forbidden</code>
Disabled user uses an unexpired token	401

Appendix C: troubleshooting

Symptom	Fix
<code>ModuleNotFoundError: No module named 'jwt'</code>	<code>pip install PyJWT</code> . Do NOT install the package called <code>jwt</code> ; it is a different library with the same import name.
Form data requires "python-multipart"	<code>pip install python-multipart</code>
<code>ValidationError ... JWT_SECRET_KEY</code> at startup	The secret is missing or shorter than 32 characters. Check <code>.env</code> , or that you started <code>uvicorn</code> from the project folder.
<code>sqlite3.OperationalError: no such column: accounts.owner_id</code>	Your <code>bank.db</code> was created before Task 5. Stop the server, delete <code>bank.db</code> , restart.
<code>InvalidSubjectError: Subject must be a string</code>	Use <code>"sub": str(user_id)</code> and <code>int(claims["sub"])</code> when reading it back.
422 from <code>/auth/token</code>	You sent JSON. The OAuth2 password flow needs form data: <code>-d "username=..&password=.."</code> .
Every Extension 5 test errors in setup	Normal before Tasks 1–2: the <code>client</code> fixture cannot log in yet.
<code>AttributeError: 'NoneType' ... hashed_password</code> in login	You used <code>user.hashed_password</code> before checking <code>user</code> exists. Use the <code>DUMMY_HASH</code> pattern.